Homework 5: Scheduling and Grow B Prep (Group) Due: Nov. 2, 11:59PM

Learning Objectives

- Use CSP to optimize resource usage for job-shop scheduling
- Use CSP to implement a scheduler for the TerraBot
- Implement a dynamic scheduler that is based on agent observations
- Prepare your agent for Grow Period B

This is a group assignment consisting of four parts. You should decide with your teammates how to divide the work on the first three parts (you can have one person do each part, or you can work on all the parts together), but everyone should work together on the last part (preparing for the second grow period).

Download and unzip the assignment code from Canvas. Place the folder in `~/TerraBot/agents/`.

OrTools

This assignment makes extensive use of `ortools`. Some of the common functions you may want to use to add constraints include:

- `Add()`: used to indicate (in)equality between variables (or constants), multiplying/dividing by constants, summing variables in a list (remember that you can sum binary variables to get counts).

  You can add `.OnlyEnforceIf()` to the end of an `Add` constraint to indicate that the constraint should be applied only if the argument to `OnlyEnforceIf` is `True` (or 1). Note that the argument **`must be a simple Boolean variable`**, for instance, it cannot be a sum or a relation. If you need to, you can get that behavior by creating an intermediate Boolean variable and use the following template:

  ```
  foo = model.NewBoolVar('foo')
  model.Add(sum(z_vars) > 0).OnlyEnforceIf(foo)
  model.Add(sum(z_vars) <= 0).OnlyEnforceIf(foo.Not())
  model.Add(x == y).OnlyEnforceIf(foo)
  ```

  This is equivalent to (which `ortools` does not actually handle):

  ```
  model.Add(x == y).OnlyEnforceIf(sum(z_vars) > 0)
  ```

- `AddMultiplicationEquality()`: used to constrain one variable equal to the product of other variables (`AddDivisionEquality` works similarly).
- `AddMaxEquality()`: constrain one variable to the maximum of a list of variables (`MinEquality` works similarly).
- `AddNoOverlap()`: constrain a set of **interval variables** not to overlap in time (that is, for each interval in the list, either the start of an interval is $\geq$ the ends of all the other intervals, or the end of that interval is $\leq$ the starts of the other intervals).

  When using these constraints with `OptionalIntervals`, `ortools` does the "right" thing by enforcing these constraints only for intervals that are "present" (i.e., those that are scheduled to occur).

- `AddReservoirConstraintWithActive()`: Used to model resource usage. The function takes a list of time variables, at which points some change to the resource might occur; a list of "demands", which are the changes to the resources; a list of "actives", which are Boolean variables that indicate whether the change is actually scheduled to take place, and a min/max range of the resource.

  The idea is that, for each time, if the associated active variable (same position in the list) is `True/1`, then the associated demand is added to/subtracted from the reservoir (depending on whether the demand is positive or negative). The `AddReservoirConstraintWithActive` constraint ensures that the value of the reservoir never exceeds the range limits (i.e., is never less than the minimum and never greater than the maximum). When you set up the reservoir, you don't need to worry about the order of the lists (e.g., which time comes first) - the constraint will take care of that automatically for you. See the October 19 lecture notes for more details.

Refer to https://developers.google.com/optimization/reference/python/sat/python/cp_model for a complete list of the functions you can use to add constraints.

Part 1: Resource Utilization for Automated Factories (27 points)

In this part, you will be modifying `job_scheduler.py` by using `ortools` to schedule factory jobs while trying to maximize profit. A factory job produces an object (or objects) with a certain value, and one maximizes profit by using various resources in optimal ways to produce high-value objects. Specifically, a **job** consists of a list of tasks that must be completed in the order given. A **task** can be carried out using a **machine** and, for most tasks, there are options about which machine to use (note that different tasks might need to use the same machine). The decision about which machine to use for a task is based on the availability of the machine, the resources that it uses, the time it takes the machine to complete the task, and the value of the product produced (which relates to how much money the object can be sold for). Typically, there is an inverse relationship between time to complete the task and value – the longer it takes to complete the task, the higher the quality, and hence the higher the value of the object produced.

A task typically requires a set of **tools** and a set of **parts** to complete the task. Tools are in use during the task, but then get returned to the pool (they are reusable resources); Parts are used up by the task, but there may be jobs that can produce additional parts (they are renewable resources).

There is also a **deadline** for completing the various tasks (e.g., an 8-hour shift at the factory). It is often the case that not all requested jobs can be completed within the allotted time – part of the scheduling problem is to determine which jobs to schedule and which to leave out.

We have defined a number of classes in `job_scheduler.py` to aid in this assignment:

- `Machine`: represents a factory machine that can be used to make things. A `Machine` object has a `name` and its `energy_cost` per hour (some machines are more efficient than others).
- `Tool`: represents a pool of tools that can be used by tasks, such as some number of wrenches. A `Tool` object has a `name` and the number of tools (`num`) available in the pool.
- `Part`: represents a pool of parts that are needed to produce objects. A `Part` object has a `name`, the `quantity` of parts initially available in the pool, and the `cost` of using one of the parts.
- `Task`: represents part of the sequence needed by a job to make objects. A `Task` object has a `name`, a list of `tools` that it requires, and a list of `parts` it requires. Each tool in the list needs to be available throughout the duration of the task. Each part in the list needs to be available at the start of the task and is used up by the task. If a task needs multiple tools or parts of the same type, they are repeated in the list (e.g., ['wrench', 'hammer', 'wrench'] indicates the need for two wrenches and a hammer). The **task_machines** attribute is a list of `TaskMachine` instances (see below) that indicate which machines can achieve those task.
- `PartsTask`: represents a task that can create new parts, which can then be used by other tasks that need that type of part. It is a subclass of `Task`. In addition to the `name, tools,` and `parts` required for a `Task`, a `PartsTask` object specifies which `part` is produced and the `quantity` that is produced at one time (i.e., executing the task once may produce multiple parts).
- `TaskMachine`: represents the fact that a machine can be used to complete some task. A `TaskMachine` specifies the `Task` and `Machine` objects, the `duration` the machine takes to complete the task (given by an integral number of hours), and the `value` of the product produced.
- `Job`: Represents the tasks needed to create some object. A `Job` specifies the `name` of the job and a list of `Task`s, all of which need to be done, in the order given, to produce the object. Note that the value of a job is equal to the sum of the values for all the tasks that comprise that job.

For example, one might have job `J1` consisting of tasks `T1` and `T2`. `T1` needs tool `Tool1` and parts `Part1` and `Part2`, and can be done by either machine `M1`, taking one hour and producing a value of 200, or by `M2`, taking 2 hours and producing a value of 300. Task `T2` needs no tools and two instances of part `Part3` and can be done by either machine `M2`, taking 3 hours and producing a value of 500, or by `M3`, taking 2 hours and producing a value of 250. Thus, there are 4 choices for completing `J1`: 1) using `M1` and `M2`, taking 4 hours with a value of 700; 2) using `M1` and `M3`, taking 3 hours with a value of 450; 3) using `M2` followed by `M2` again, taking 5 hours with a value of 800; and 4) using `M2` and `M3`, taking 4 hours with a value of 550. Which is best depends on what other jobs need to be scheduled and how much energy each machine uses (since electricity usage reduces profit).

We have also specified sets of variables that will likely be useful in what follows (see the function `create_job_task_variables` in `job_scheduler.py`). Specifically:

- `starts`: is a dictionary whose keys are a list of `[job, task, machine]`. The values are integer `ortools` variables representing the start times for doing the task on the particular machine for that job.
- `ends`: is a dictionary whose keys are a list of `[job, task, machine]`, and whose values are integer `ortools` variables representing the end times for doing the task on the particular machine for that job.
- `scheduleds`: is a dictionary whose keys are a list of `[job, task, machine]`, and whose values are Boolean `ortools` variables indicating whether that machine was actually scheduled to do the task for that job.
- `intervals`: is a dictionary whose keys are a list of `[job, task, machine]`, and whose values are `OptionalInterval` variables that encode the start, end, and duration of a task being performed by the machine. All intervals are constrained to start at, or after, time 1 and end no later than the deadline.

Note: for consistency, use the function `self._key(job, task, machine)` to generate keys to access the above dictionaries.

The following steps create your constraint model one step at a time. At each step, you can test your model using `python autograder.py -p 1 -s <n>`, where `n` is the step number. All constraints up to, and including, that step will be added to test your solution. You can also use the `-v` flag to generate more verbose output and the `-g` flag to visualize the schedule (although for some of the steps, where the constraints are not fully developed, visualization may not work correctly).

Step 1: Task Constraints (2 points)

The first step is to implement `create_task_constraints` to encode the constraint that, for each job, each task much be achieved by only one machine. Recall that the definition of tasks include a list of `TaskMachine` objects that indicate what machines can be used to achieve that task. **Note**: Be aware that the job may not end up being scheduled at all.

Test using `python autograder.py -p 1 -s 1`
Note: If you use the `-v` option, it will print out your solution; If you use the `-g` option, it will display your solution graphically (you can use these options for all the subsequent steps, as well).

Step 2: Machine Constraints (2 points)

The next step is to implement `create_machine_constraints` to encode the constraint that each machine can handle only one task at a time. The implementation will be a bit different from the classroom scheduling exercise you did in class, since here we are using the interval model, rather than the binary model, but the lecture slides should help.

Test using `python autograder.py -p 1 -s 2`, which will also include the task constraints from Step 1.

Step 3: Task Ordering Constraints (3 points)

This constraint is a bit more involved. **You need to implement** `create_task_ordering_constraints` to encode that all the tasks of a given job are done in sequence – for instance, whichever machine is used for the first task in a job, it must end before the machine chosen for the second task in that job begins. Recall that the `scheduleds` variables indicate whether a given machine has actually been scheduled to achieve a task for a job.

Note that the list of `task_machines` is a `TaskMachine` instance (in past years, many students initially thought they were `Machine` instances). Also, this may be a good place to use `OnlyEnforceIf`.

Test using `python autograder.py -p 1 -s 3`

Step 4: Task Completion Constraints (3 points)

This constraint is also a bit more involved. **You need to implement** `create_task_completion_constraints` to encode that if any job is started it must be finished by the deadline. That is, either *all* tasks in a job are scheduled (more accurately, the machines scheduled to achieve the tasks), or *none* of them are.

Test using `python autograder.py -p 1 -s 4`

Step 5: Tool Constraints (5 points)

Now we get to resource usage! **For this step, implement** `create_tool_constraints` to encode how tools are used by the factory. That is, at the start of a scheduled task the tool is removed from the pool and at the end it is returned. Use the `AddReservoirConstraintWithActive()` constraint to ensure that the number of tools in concurrent use is never greater than the pool size. See the lecture slides from October 19 if you need help on this part.

Test using `python autograder.py -p 1 -s 5`

Step 6: Part Constraints (5 points)

Continuing with resource usage, **implement** `create_part_constraints` to encode how parts are used by the factory. That is, if a scheduled task needs a part, it is removed from the pool of parts at the **start** of the task (and is not returned at the end – it is consumed). In addition, if a scheduled `PartsTask` task creates a part, the quantity of that part produced is added to the pool at the **end** of the `PartsTask` task (you can determine if a task is a `PartsTask` with the function `task.isPartsTask()`). Use the `AddReservoirConstraintWithActive()` constraint to ensure that the number of parts never falls below zero. **As a extra constraint, assume that the number of parts in the pool can never be greater than the number at the start** – this encodes that there is some storage limit for parts. See the lecture notes from October 19 if you need help on this part.

Test using `python autograder.py -p 1 -s 6`

Step 7: Maximizing Profit (5 points)

Up until now, the optimization criterion was to maximize total value (implemented for you in `add_values`). For this step, you will maximize net profit, which is defined as the **value** of the products minus the **cost** of producing them. There are two costs that need to be taken into account:

- Each machine uses energy to produce the parts (some machines are more efficient than others). One part of the costs is the total cost of the energy used by the machine over the duration that it is working (note that the `energy_cost` attribute of a `Machine` is the energy usage **per hour**).
- Tasks may need to use parts in the manufacturing process (the `Task` object specifies the list of parts it needs to use). Those parts have a cost, specified by `part.cost` that must be accounted for in the cost function.

Fill in the `add_costs` function to set the `ortools` variable `self.cost` to the total cost of production (the sum of the two costs above). The function `add_optimization` will then define an objective function that maximizes net profit (you don't need to alter `add_optimization`). **Hint:** if you multiply some equation by a `scheduleds` variable, the result will be zero if the variable is `False` (0) and the value of the equation if the variable is `True` (1).

Test using `python autograder.py -p 1 -s 7`

Step 8: Testing Everything Together (2 points)

When you are all done, run `python autograder.py -p 1`, and it will run all the tests for this part. In particular, it will test whether constraints you added in later steps somehow messed up earlier solutions. It will also test a larger range of problems – some of which will take a significant amount of time to run. Don't forget that you can use the `-v` and `-g` options to get more insight into what your code is producing.

Part 2: Creating a Greenhouse Scheduler (28 points total)

In this part, **you will modify** `greenhouse_scheduler.py` by using `ortools` to create a program that will schedule each of your behaviors. We present below four steps that will guide you through a way to write the scheduler, but you are free to delete all our code in `greenhouse_scheduler.py` and create your own, if you wish.

We have implemented the `GreenhouseScheduler` class that has the basic functions for this part, including creating the necessary constraint variables in the `__init__` function. In particular, `self.all_jobs[behavior,time]` is a Boolean variable indicating whether a behavior is enabled at interval `time`, where `self.minutes_per_chunk` represent how many minutes are in a block of `time` and `self.horizon` represents how many chunks of time there are in a day. If not `None`, `self.sched_file` is the name of the file where the generated schedule will be saved. Finally, `self.behaviors_info` is a dictionary, where the keys behaviors and the values are instances of the `BehaviorInfo` class where:

- `min_extent`: The *least* cumulative amount of time, in minutes, that all scheduled behaviors of that type need to be run.
- `night_extent`: The maximum amount of time, in minutes, the behavior should run at night between hours [20,24) and [0,8), that is, starting from 8pm until just before 8am.
- `min_spacing`: The minimum spacing between the behaviors, in minutes. Specifically, no two behaviors can be scheduled for closer than `min_spacing` minutes. Note that a time of 0 means that behaviors can be scheduled back-to-back.
- `max_spacing`: The maximum spacing between the behaviors, in minutes. Specifically, at least one behavior must be scheduled in every block of `max_spacing` minutes. Note that this constraint and the previous one do not wrap around midnight (i.e., the constraints do not apply from the end of one day to the beginning of the next).

For instance, if behaviors are enabled for 30 minutes at a time (based on `self.minutes_per_chunk`), then:

- `behavior_info['LowerHumidBehavior'].min_extent == 480` means that the 'LowerHumidBehavior' behavior needs to be run for at least 480 minutes (16 thirty-minute blocks) per day;
- `behavior_info['LowerHumidBehavior'].night_extent == 300` means that 'LowerHumidBehavior' behaviors can be scheduled for at most 10 thirty-minute blocks of time at night (8pm to 8am);
- `behavior_info['LowerHumidBehavior'].min_spacing == 60` means that instances must be separated by at least 2 blocks; and
- `behavior_info['LowerHumidBehavior'].max_spacing == 120` means that at least one instance of the behavior must be scheduled every 4 blocks.

For each step below, the autograder uses 5 test cases, the first checks to see whether your model's constraints are feasible (that is, they produce *some* solution) and the second checks to see whether they produce no solution if the constraints are infeasible. The final three tests check whether you are producing a solution that is consistent with the refsol's constraints. Note that you'll get a few points for not doing anything, since the baseline code produces no schedule and that's a correct answer for one of the tests.

Step 1: Implementing Duration Constraints (6 points)

For this step, you will implement the duration constraints of each behavior (that is, how many instances are scheduled) **by filling in the function** `createDurationConstraints`**.**

For this, you should loop through each of the behaviors in `behavior_info` and make sure that the sum of all the times that behavior is scheduled to run is at least the time required by `min_extent`. Make sure to convert the specified the time in minutes into the number of required time blocks using `self.minutes_per_chunk`.

You can test this with `python autograder.py -p 2 -s 1`
Note: If you use the `-v` option, it will print out your solution; If you use the `-g` option, it will display your solution graphically (you can use these options for all the subsequent steps, as well).

Step 2: Mutually Exclusive Constraints (6 points)

For this step, you will implement the mutually exclusive constraints between behaviors (that is, behaviors that need the same resource should not be scheduled at the same time), **by filling in the function `createMutuallyExclusiveConstraints`.**

For this, you should loop through each time block from 0 until the time horizon and make sure that none of the mutually exclusive behaviors are enabled at the same time. A behavior is mutually exclusive with another behavior if it shares the same actuator, if one behavior wants the actuator on and the other off, or if it is the Raise and Lower for the same enviromental value (temperature, humidity, moisture). A full list is provided as a comment in the function. `Hint:` A simple way to do this is to simply include all the pairwise mutually exclusive constraints.

You can test everything so far with `python autograder.py -p 2 -s 2`

Step 3: Night Constraints (6 points)

For this step you will implement the night constraint of each behavior (that is the maximum amount of time the behavior can operate at night) **by filling in the function `createNightConstraints`.**

For this, you should loop through each of the behaviors in `behavior_info` and make sure that the sum of all the times that behavior runs during the night is less than or equal to the time allowed as specified by `night_extent`. Make sure to convert the time in minutes into the number of required time blocks using `self.minutes_per_chunk`.

Note: "Night" is considered to run from 8pm *up to* 8am; since this wraps at midnight, you need to combine the blocks from 8pm to midnight with those from midnight up to 8am.

You can test everything so far with `python autograder.py -p 2 -s 3`

Step 4: Spacing Constraints (9 points)

For this step, you will implement the min and max spacing constraints of each behavior by **filling in the function `createSpacingConstraints`.**

For this, you should loop through each of the behaviors and add constraints that ensure there is at least the minimum time between scheduled instances (`min_spacing`) and that there is at least one instance scheduled in every maximum block of time (`max_spacing`).

A suggested approach is to first convert the min and max spacing values to blocks of time `min_b` and `max_b` (using `self.minutes_per_chunk`). Then, for all windows of `min_b+1` consecutive blocks, **at most** one instance of the behavior can be scheduled, and for all windows of `max_b+1` blocks **at least** one instances must be scheduled. If a behavior is not run at night (`night_extent == 0`), create min/max windows for that behavior only during the day (8am to 8pm). Make sure to convert the times in minutes to the desired number of time blocks. Finally, note that the windows at the end of the day may be smaller than the min/max times (e.g., a window starting at 11pm can last for only an hour). You don't have to worry about the max constraint for blocks of time that would extend to after either midnight or 8pm, depending on whether the behavior runs at night.

You can test everything with `python autograder.py -p 2 -s 4`

Step 5: Testing Everything Together (1 point)

When you are all done, run `python autograder.py -p 2`, and it will run all the tests for this part. In particular, it will test whether constraints you added in later steps somehow messed up earlier solutions. Don't forget that you can use the `-v` and `-g` options to get more insight into what your code is producing.

Part 3: Dynamic Scheduler (30 points)

Up until now, your agent has been running a predefined schedule (`greenhouse_schedule.txt`). That schedule was created based on our intuition of how often the behaviors would need to run, not using any actual data. For this part, you need to develop an approach that adapts the schedule to historical conditions. This can take any number of forms — we offer two suggested approaches below, but feel free to develop your own approach.

Ideas:

1. Plants have different needs depending on their stage of growth, especially how much light and moisture they need/can tolerate. Do some research to determine what radishes and lettuce need when they first germinate, start growing, and get to the point where they are starting to mature (after about two weeks). Use vision to determine what stage your plants are at and have the schedule change the frequency that the lighting and watering behaviors are enabled. Also use the insolation prediction that you developed previously to monitor and maintain the appropriate amount of light during the day. Based on these values, you can adjust the next day's schedule such that the daily light and moisture average out appropriately.
2. You will notice that some behaviors rarely need to be enabled (e.g., lowering temperature rarely has to run, especially in the evening) and some behaviors may need to be run more frequently. Implement monitors to determine when behaviors are actually needed (e.g., they turn actuators on) and when they should be needed (e.g., some sensor limit is reached when the associated behavior is not running). Use this data, say averaged over a day, to change the scheduled frequency of behaviors. You don't have to do this for all the behaviors – pick two that you think would most benefit from this rescheduling.

To do the rescheduling, you can use the scheduler developed in Part 2 or, to help you, we have provided a a pre-compiled scheduler that can be accessed using `greenhouse_scheduler_ref.py`. Read the introduction of Part 2 to see how to define the arguments to the `GreenhouseScheduler` class.

No matter which approach you choose, you will need to:

- In a file of your own choosing, create a `Monitor` to observe and compute the values you need, and have it update the `behavior_info` parameters needed for the scheduler (see the description in Step 2).
- Given those updated parameters, create an instance of `GreenhouseScheduler` and invoke its `solveProblem` method to create a schedule based on the modified constraints, which in turn are based on observed conditions. See the example code at the end of `greenhouse_scheduler.py` for how to invoke things.
- You may find that `solveProblem` indicates that the constraints are infeasible. In that case, your agent will have to modify the parameters to allow it to optimize the behaviors you care about.
- Call the planner functions `setTestingSchedule` and `switch_to_test_sched` (defined in `layers.py`) to set and load a testing file and run from that (rather than having it read the txt file).

Once you have implemented (and tested) your dynamic scheduler, compare it with the baseline schedule. In particular, **run your agent for 2 (simulated) weeks each using both your dynamically produced schedules and the standard `greenhouse_schedule.txt`. Based on the results, write a report (of approximately 4-5 single-spaced pages) that includes:**

- **The approach you took**: Include how you implemented both the monitor for changing the `behavior_info` and the procedure for finding an optimal policy based on those changes (e.g., how you handled infeasible schedules);
- **Effects on the schedule**: In particular, capture 2-3 schedules and visualize them using the function `plot_binary.py` in `visualize_solution.py` (look at `autograder.py` for an example of how to use that function). Identify salient differences from the baseline schedule and discuss why those changes occurred, based on your dynamic approach.
- **Effects on plant growth**: Compare and contrast the plant growth from the baseline schedule and your dynamic scheduler. Capture your estimated plant health or other quantities important for your scheduling. Feel free to capture some visualizations of your plants in both cases to compare as well.

Part 4: Grow Period B Preparation (15 points total)

In this part, you will prepare for Grow Period B by improving both your agent's behaviors and the email behavior.

Step 1: Improving Behaviors (7 points)

Based on what you observed with your agent's performance in Grow Period A, there are likely to be aspects of the behaviors that need to be improved. In particular, many of the agents used significantly less water than was expected. So, one potential focus would be to implement a new, more reliable watering behavior. You can use **any method** that you want, although the emphasis should be on watering enough, but not too much. **We will give you only 1500 ml for the 2-week grow period, so make sure you don't use it all up in the first week!**

Some things to consider are how much to trust the moisture sensors – look at the logs from Grow Period A and analyze how the moisture sensor values changed over time. Think about how the weight sensors can be used to estimate moisture, and how you could incorporate those. Think about how the requirements for water change over time – as the plants get bigger (which you can tell from your vision module) they need more water.

You should also update any sensor limits for determining when behaviors should turn actuators on and off, based on the observed behavior from Grow Period A or any insights your peers have made about their agents. For instance, if you are continuing to use the moisture sensors, what limits should be used? What insolation, humidity, and temperature levels are best for radishes and lettuce (based on research you do). Note that if you used the insolation monitor in Grow Period A with the default insolation level from the homework assignment, that was probably too low for good plant growth.

You may also want to consider incorporating the dynamic scheduler, based on the simulation results obtained in Part 3. This could yield good results on the real hardware, since it can adapt the schedule to real-world conditions.

Write a report (of approximately 2-3 single-spaced pages) that describes the changes you made to the behaviors. The report should include technical details of what the changes were and why you decided to make those changes (you may want to include graphs indicating what made you decide to change any limits).

Step 2: Informative Emails (8 points)

For this grow period, your emails should all be human (e.g., professor) readable! To be "readable", we want at most 2 decimal places, data clearly organized between raw and average, titles in all caps, appropriate spacing, etc. Each of the items below should have some sort of title associated with them. Additionally, for values such as temperature, you should have units associated with it. For other items, such as a watering summary, the sentences or other output should be clear.

At a minimum, the daily emails should:

- Have the time of the report be something more readable. We suggest using the date and how many days it has been running for (look at the function `clock_time` in `terrabot_utils.py`);
- Include a summary of watering over the last 24 hours. We would like to know whether your agent watered and, if so, how much water was used. Ideally, you would also include an explanation for why the agent did, or did not, water during the previous 24 hours;
- Include a summary of daily insolation, based on the insolation monitor from the Monitoring and Testing assignment **Note: Report on the *previous day's total insolation*, not what it is at the time you are sending the email!**;
- Include the two images returned from the function `foliageImages` in `vision.py` from Part 5, Step 1 of the Computer Vision assignment. In particular, one image should have the measuring stick outlined and a line at the plant height and the other image should show the detected foliage;

- Include the assessment of plant health as determined by the function `plantHealth` from Part 5, Step 2 of the Computer Vision assignment. In addition to indicating how the plants are doing that day, indicate whether they are doing better or worse than the previous day; and
- Include a summary of how the weight of the pan changed since the previous day. Estimate how much of that change was based on watering, and how much based on plant growth.

Feel free to add more sensor data in the daily emails, including graphs of sensor data over time. Include whatever you think is important in order to better understand how your agent is performing and how the plants are doing. Include the raw sensor data only if you think it will be informative.

Once you have designed and tested your emails (please don't send test emails to the professor or the TAs!), run it on the TerraBot hardware. **Submit a screen shot of the email** that illustrates all the above points. Feel free to submit screen shots to us before the assignment deadline so we can tell you if what you have done is sufficient to get full credit on this part of the assignment.

Step 3: Testing (0 points)

In addition to these modifications, as always, you should be testing your agent on the real hardware. In particular, test your updated email behavior (especially its use of the computer vision) and, if you decide to use the dynamic scheduler, be sure to test it thoroughly prior to the start of the grow period. Work as a team to make sure your sensors and actuators are working, and either reach out to us if you think something is wrong, or adjust the limits accordingly. It is always possible that something can change or break as we set up the new grow period (we will give you a day or two after the seeds are planted before you will have to launch your agent). Especially test thoroughly all the new features added in this part.

Since you will be running the foliage classifier and plant height measurement as part of Grow Period B, it is important for those to work well on your own TerraBots. To that end, run your classifier on the images taken during Grow Period A and look to see whether your classifier is doing well (you'll have to estimate by looking at the images overlaid with the foliage mask). If your classifier is doing poorly, you may have to retrain it to manually annotating the images to select the foliage (I used Roboflow – https://roboflow.com/ – for the images in the computer vision assignment; it was relatively straightforward and, if you want, I can share the code to convert from the Roboflow output to mask files). Note that you will also need to identify the stick mask, but for that wait until the grow period has started, since the camera will likely move once we plant for Grow Period B.

While this part is not worth any points for this assignment, careful testing will ensure that your agent will run fault-free for the two week Grow Period. We will monitor your agents to determine how often they are restarted, and will apply a penalty for each restart.

Submission

Submit the following files to Canvas: `greenhouse_scheduler.py, job_scheduler.py`, your report for Part 3 on the dynamic scheduler, your report for Part 4, Step 1 on changes you made to behaviors, and screen shots of your improved email behavior from Part 4, Step 2.

When added to a directory with the rest of the agent files, your code should run without error.